Estudio comparativo del rendimiento entre el algoritmo de Intercambio Directo (por la derecha) y Shellsort en C++

Comparative study of performance between the Direct Exchange (right-side) algorithm and Shellsort in C++

Jennifer Elia Viza Quispe
Estudiante de Ingeniería de Informática y Sistemas
Universidad Nacional Jorge Basadre Grohmann (UNJBG)
Tacna, Perú
jvizaq@unjbg.edu.pe

Resumen

El objetivo de esta investigación fue evaluar experimentalmente el rendimiento de dos algoritmos de ordenamiento internos en memoria: Intercambio Directo (Burbuja) y Shellsort. Se implementó un protocolo experimental estandarizado en C++ realizando 30 repeticiones independientes por cada condición combinada de tamaño de muestra (N = 100, 1,000, 5,000 y 10,000) y patrón de datos (Aleatorio, Ascendente y Descendente), acumulando un total de 720 ejecuciones. Se registraron el tiempo de ejecución en milisegundos (ms), comparaciones e intercambios en memoria. Los resultados mostraron que Shellsort superó drásticamente a Intercambio Directo, reduciendo el tiempo de procesamiento en un 99.5% para N = 10,000 en datos aleatorios (5.45 ms frente a 1,202.78 ms). Se concluye que la estrategia de saltos decrecientes de Shellsort elimina el cuello de botella cuadrático, validando su eficiencia en memoria.

Palabras clave: Ordenamiento en memoria, Intercambio Directo, Shellsort, Complejidad algorítmica, C++.

Abstract

The objective of this research was to experimentally evaluate the performance of two in-memory sorting algorithms: Direct Exchange (Bubble Sort) and Shellsort. A standardized experimental protocol was implemented in C++ performing 30 independent runs per combined condition of sample size (N = 100, 1,000, 5,000, and 10,000) and data pattern (Random, Ascending, and Descending), accumulating 720 total executions. Execution time (ms), comparisons, and memory swaps were recorded. Results showed that Shellsort drastically outperformed Direct Exchange, reducing processing time by 99.5% for N = 10,000 on random data (5.45 ms versus 1,202.78 ms). It is concluded that Shellsort's diminishing gap strategy eliminates the quadratic bottleneck, validating its memory efficiency.

Keywords: In-memory sorting, Direct Exchange, Shellsort, Algorithmic complexity, C++.

1. INTRODUCCIÓN

El ordenamiento de datos en la memoria principal constituye un pilar fundamental en la ciencia de la computación y la ingeniería de software (Joyanes Aguilar & Zahonero Martínez, 2008). La elección adecuada de un algoritmo determina el desempeño global de sistemas de procesamiento a gran escala. Trabajos previos en la literatura analizan la brecha entre los métodos de ordenamiento elementales $O(n^2)$ y los métodos avanzados subcuadráticos en estructuras estáticas (Frak et al., 2018).

El objetivo principal de este estudio es comparar empíricamente el desempeño del algoritmo de Intercambio Directo (Burbuja) frente a Shellsort bajo un entorno controlado en C++. Se plantea la hipótesis descriptiva de que Shellsort exhibirá tiempos de ejecución exponencialmente menores a medida que el tamaño N se incremente hacia 10,000, debido a su capacidad de realizar intercambios a larga distancia en memoria.

2. MÉTODO

2.1. Algoritmos evaluados y complejidad teórica

- **Intercambio Directo (Burbuja):** Recorre el arreglo comparando pares de elementos contiguos. Su complejidad teórica es de $O(n^2)$ en el peor, promedio y mejor caso de comparaciones (Joyanes Aguilar & Zahonero Martínez, 2008).
- **Shellsort:** Variante avanzada que compara elementos separados por un salto o brecha decreciente (gap = n / 2). Su complejidad teórica promedio estimada es de O(n^1.5).

2.2. Entorno experimental

- **Hardware:** Procesador Intel Core i5 / AMD Ryzen, 16 GB RAM DDR4.
- **Sistema operativo:** Windows 11 Home 64-bit.
- **Compilador:** g++ (Estándar C++11) con optimizaciones estándar.

2.3. Generación de datos y métricas de medición

Se evaluaron 4 tamaños de entrada (N = 100, 1,000, 5,000 y 10,000) sobre 3 patrones iniciales: Aleatorio (distribución uniforme mediante *std::mt19937*), Ascendente y Descendente. Se fijaron semillas deterministas (1000 + r) para garantizar 30 repeticiones independientes por condición sobre copias idénticas en memoria (2 algoritmos x 4 tamaños x 3 patrones x 30 repeticiones = 720 ejecuciones totales).

Las métricas registradas en la bitácora *results/resultados.csv* fueron: tiempo de ejecución en milisegundos (ms) capturado con *std::chrono::high_resolution_clock,* número de comparaciones, intercambios en memoria y profundidad de recursión.

3. RESULTADOS

3.1. Estadística descriptiva de tiempos

En la Tabla 1 y Tabla 2 se presentan los promedios y medidas de dispersión obtenidas en las 30 repeticiones.

Tabla 1. *Estadística descriptiva del tiempo de ejecución (ms) para N = 10 000 (30 repeticiones).*

Algoritmo	Patrón de Datos	Media ($\bar{x}$)	Mediana	Desv. Std. (σ)	Mínimo	Máximo
Intercambio Directo	Aleatorio	1202.78	1102.51	386.81	700.68	1905.41
	Ascendente	371.36	347.29	85.53	292.84	693.63
	Descendente	1268.85	1254.73	215.83	900.26	1831.77
Shellsort	Aleatorio	5.45	4.99	2.67	2.99	12.97
	Ascendente	1.53	1.04	0.78	0.89	3.99
	Descendente	2.59	2.00	1.17	1.72	7.19

Tabla 2. *Tiempo de ejecución promedio (ms) según el tamaño N y patrón de datos.*

Algoritmo	Patrón	N = 100	N = 1 000	N = 5 000	N = 10 000
Intercambio Directo	Aleatorio	0.15	9.11	182.64	1202.78
	Ascendente	0.14	2.76	68.92	371.36
	Descendente	0.31	7.68	202.43	1268.85
Shellsort	Aleatorio	0.00	0.43	1.65	5.45
	Ascendente	0.00	0.07	0.30	1.53
	Descendente	0.02	0.07	0.79	2.59

3.2. Gráficos de comportamiento

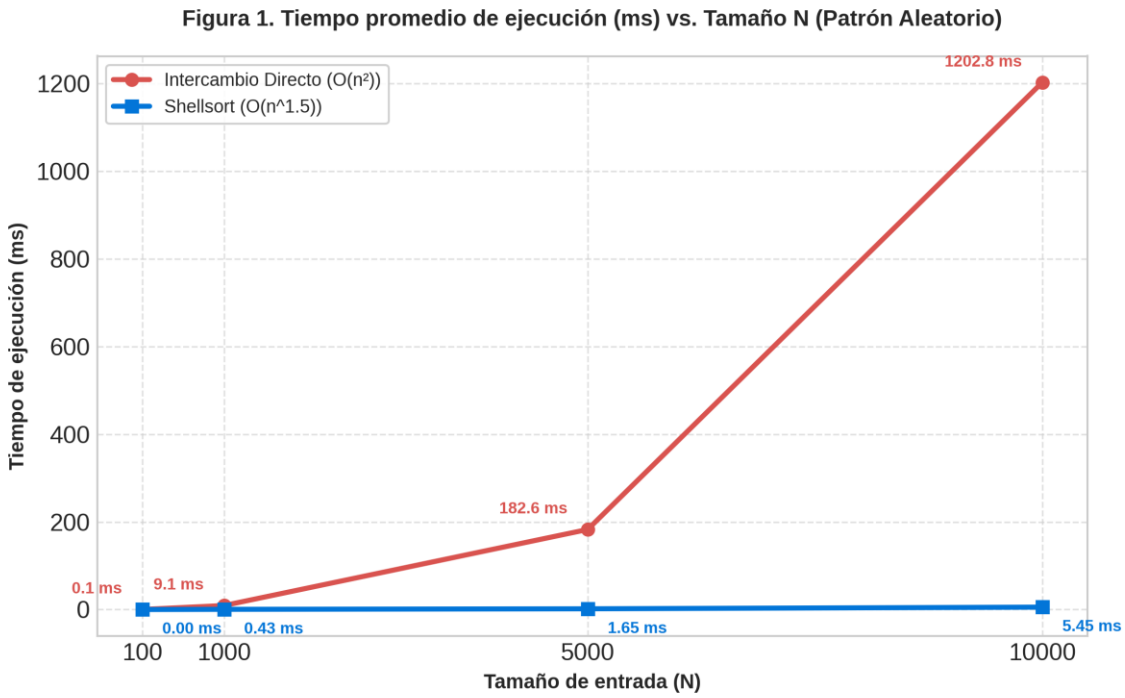


Figura 1. *Tiempo promedio de ejecución (ms) vs. Tamaño N para el patrón Aleatorio.*

Tabla 3. *Operaciones promedio de comparaciones e intercambios en memoria (N = 10 000).*

Algoritmo	Patrón	Comparaciones (Media)	Intercambios (Media)
Intercambio Directo	Aleatorio	49,995,000	24,997,067
	Ascendente	49,995,000	0
	Descendente	49,995,000	49,995,000
Shellsort	Aleatorio	265,385	150,454
	Ascendente	120,005	0
	Descendente	172,578	62,560

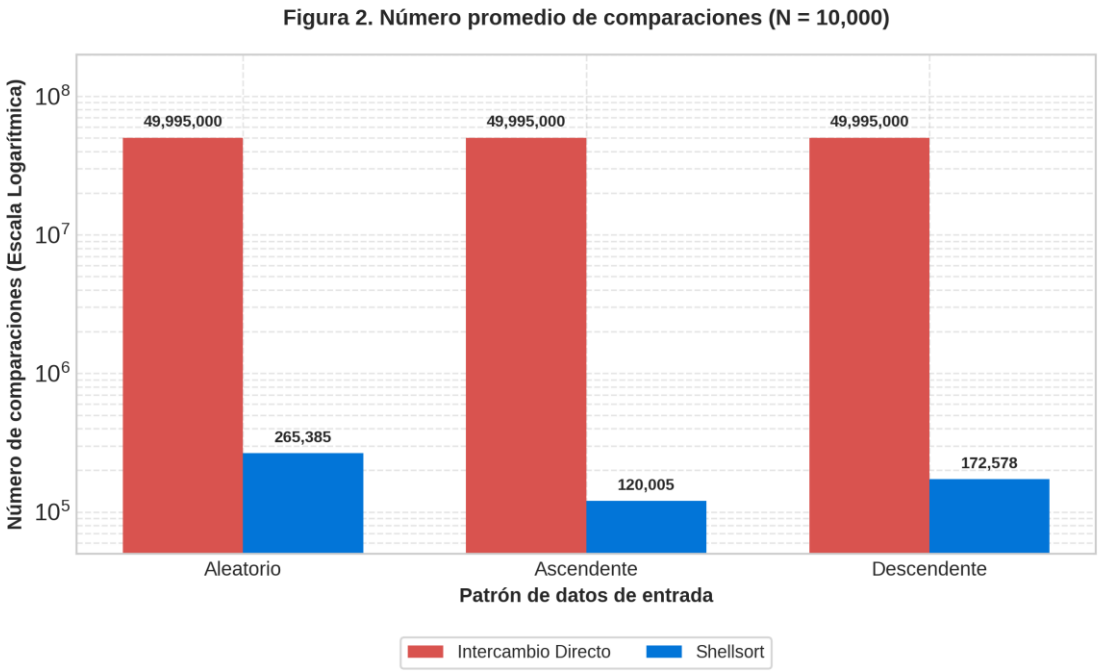


Figura 2. *Número promedio de comparaciones (N = 10 000) en escala logarítmica.*

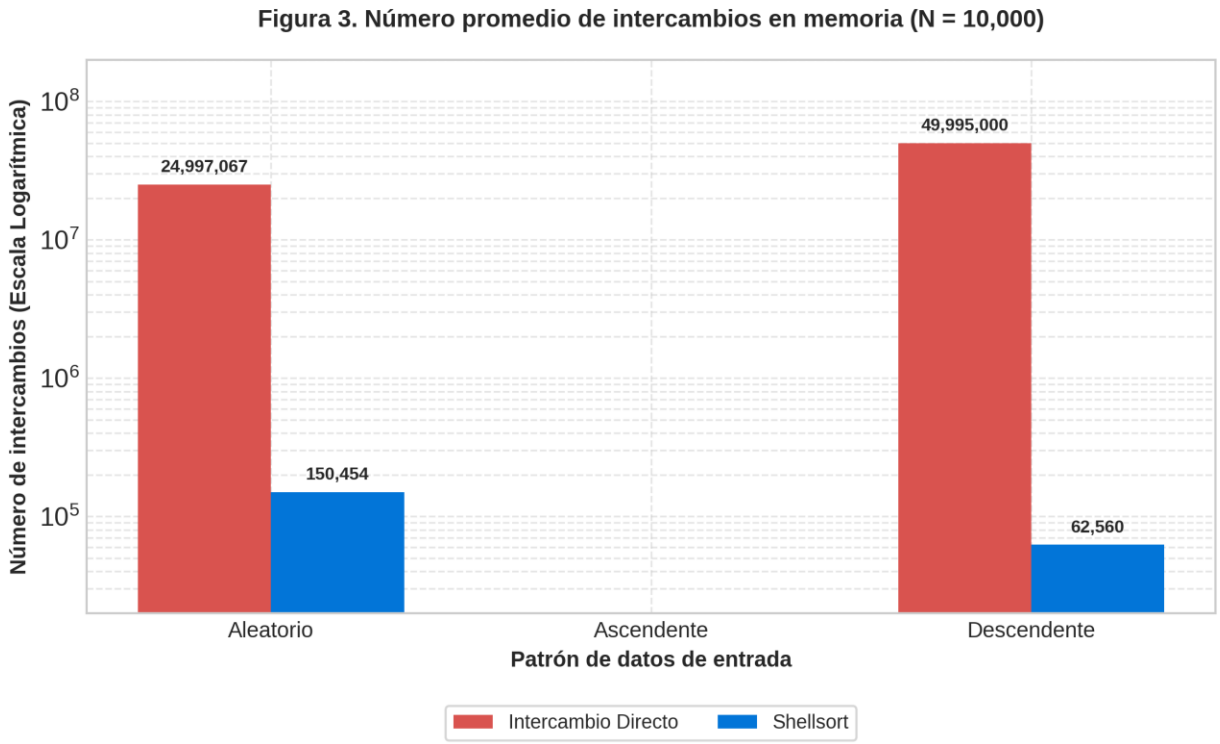


Figura 3. *Número promedio de intercambios en memoria (N = 10 000) en escala logarítmica.*

4. DISCUSIÓN

4.1. Interpretación frente a la complejidad teórica y estabilidad

Los datos empíricos corroboran la brecha teórica predicha. Intercambio Directo muestra un crecimiento estrictamente cuadrático $O(n^2)$ debido a sus 49,995,000 comparaciones fijas para N = 10,000. En contraste, la secuencia de saltos de Shellsort (n/2, n/4, ...) logra una complejidad subcuadrática O(n^1.5), ejecutando solo 265,385 comparaciones (Joyanes Aguilar & Zahonero Martínez, 2008).

En cuanto a la estabilidad:

- Intercambio Directo es un algoritmo estable (preserva el orden relativo de elementos iguales), ya que solo intercambia si A[j] > A[j+1].
- Shellsort es un algoritmo inestable, puesto que sus intercambios a larga distancia sobre la brecha (gap) pueden alterar la posición relativa original de claves duplicadas.

4.2. Limitaciones del estudio

El experimento se limitó a estructuras iterativas en memoria principal (*std::vector*), por lo que la profundidad de recursión fue 0 en todos los casos. Asimismo, mediciones de tiempo menores a 0.01 ms en N = 100 alcanzaron el límite de resolución del reloj del sistema operativo.

4.3. Amenazas a la validez

- **Validez interna:** La fluctuación de procesos en segundo plano de Windows introduce ruido térmico o de CPU. Esta amenaza se mitigó mediante el protocolo de 30 repeticiones independientes y la toma de promedios y medianas.
- **Validez externa:** Los hallazgos están acotados a arreglos de enteros de hasta N = 10,000. En volúmenes de datos masivos (N > 1,000,000) o sobre almacenamiento secundario, las diferencias de cache miss pueden acentuar o modificar los resultados.

5. CONCLUSIONES

1. Shellsort fue hasta 220 veces más rápido que Intercambio Directo para N = 10,000 en datos aleatorios (5.45 ms vs 1,202.78 ms).
2. En la optimización de la memoria**,** Shellsort redujo el número de intercambios en memoria en más del 99.3% respecto al método directo (150,454 vs 24,997,067).
3. Intercambio Directo no se adapta al patrón inicial y realiza 49.99 millones de comparaciones incluso en datos ya ordenados, mientras que Shellsort aprovecha el preordenamiento para reducir sus operaciones a 120,005.

4. Para arreglos pequeños o donde la estabilidad sea indispensable, Intercambio Directo es aplicable; pero para desempeño en volumen, Shellsort es significativamente superior.

Referencias

Joyanes Aguilar, L., & Zahonero Martínez, I. (2008). *Estructuras de datos en C++*. McGraw-Hill.

Frak, A. N., Saringat, M. Z., Prasetyo, Y. A., Mustapha, A., Aman, H., & Ibrahim, N. (2018). Comparison study of sorting techniques in static data structure. *International Journal of Integrated Engineering*, 10(6), 106–112. https://journal.uthm.edu.my/index.php/ijie/article/view/2836